LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: K.Vinay

Roll No.: A24126552086

Date: 31-08-2026

---

1. TITLE

Implementation of an Object-Oriented Student Profile Viewer using JavaScript Classes

2. AIM

To build a dynamic student profile viewer utilizing JavaScript ES6 classes to encapsulate student data and dynamically render the information to the webpage upon user interaction.

3. OBJECTIVE

The objectives of this experiment are:

- To understand and implement JavaScript ES6 Classes and constructors.
- To instantiate objects with specific property values.
- To handle click events to trigger data rendering.
- To dynamically generate HTML elements to display object properties.

4. THEORY

ES6 introduced classes in JavaScript, providing a clearer syntax for object-oriented programming. A class can define a constructor to initialize object properties. These properties can then be accessed and displayed on the web page dynamically by manipulating the DOM when an event, such as a button click, occurs.

5. ALGORITHM / PROCEDURE

1. Create the base HTML structure with a button and an empty profile container.
2. Define a Student class in JavaScript with a constructor.
3. Instantiate a student object with the user's details.
4. Select the button and profile container elements from the DOM.
5. Add a click event listener to the button.
6. Inside the event listener, clear any existing content in the profile container.
7. Dynamically create heading and paragraph elements.
8. Assign the object's properties to the text content.
9. Append all created elements to the profile container to display the data.

6. CODE EXPLANATION

The script establishes a blueprint for objects by utilizing ES6 'class Student { ... }' syntax. The 'constructor(...)' method acts to initialize all properties of any newly created instances of the class. A distinct student object containing user data is instantiated by executing 'new Student(...)'. JavaScript's document.getElementById() is then used to select the appropriate interactive and rendering elements in the DOM. Ultimately, the 'textContent' property of dynamically created DOM elements is modified to visibly present the data inside the web browser.

7. PROGRAM

index.html

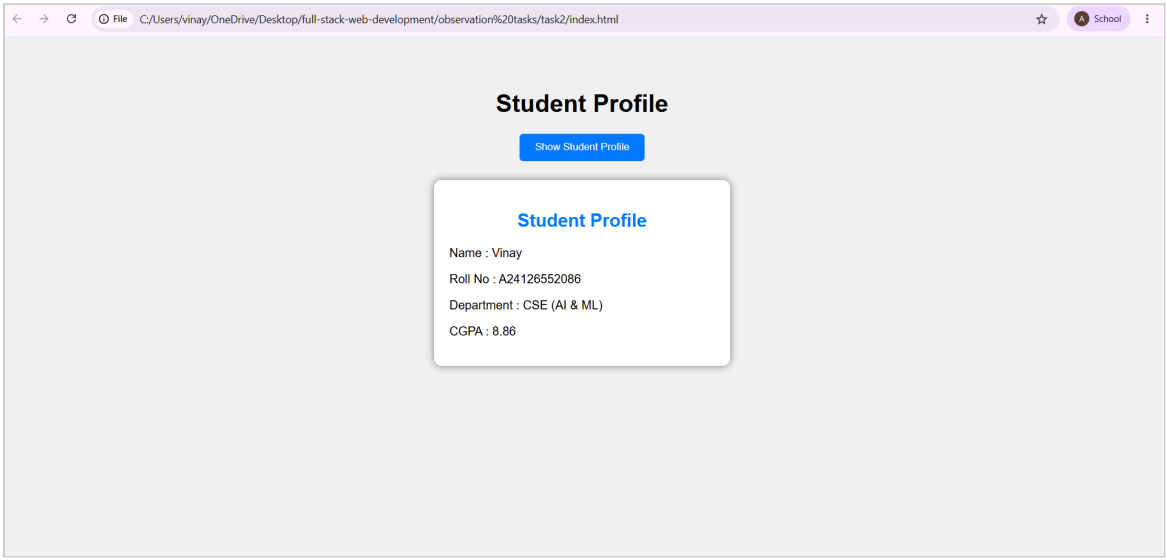
```
<!DOCTYPE html>
<html>
<head>
    <title>Student Profile</title>

    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f2f2f2;
            text-align: center;
            padding: 40px;
        }

        button {
            padding: 10px 20px;
            background-color: #007bff;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }

        button:hover {
            background-color: #0056b3;
        }

        #profile {
            width: 350px;
            margin: 25px auto;
            padding: 20px;
            background-color: white;
            border-radius: 10px;
            box-shadow: 0 0 10px gray;
            text-align: left;
        }

        #profile h2 {
            text-align: center;
            color: #007bff;
        }

        #profile p {
            font-size: 16px;
        }
    </style>
</head>

<body>

    <h1>Student Profile</h1>

    <button id="showButton">Show Student Profile</button>

    <div id="profile"></div>

    <script>
        // Student class
        class Student {
```

```
            constructor(name, rollNumber, department, cgpa) {
                this.name = name;
                this.rollNumber = rollNumber;
                this.department = department;
                this.cgpa = cgpa;
            }
        }

        // Creating Student object
        const student = new Student(
            "Vinay",
            "A24126552086",
            "CSE (AI & ML)",
            "8.86"
        );

        // DOM selection
        const button = document.getElementById("showButton");
        const profile = document.getElementById("profile");

        // Event handling
        button.addEventListener("click", function () {

            // Clear previous content
            profile.innerHTML = "";

            // Create elements dynamically
            const heading = document.createElement("h2");
            heading.textContent = "Student Profile";

            const name = document.createElement("p");
            name.textContent = "Name : " + student.name;

            const roll = document.createElement("p");
            roll.textContent = "Roll No : " + student.rollNumber;

            const department = document.createElement("p");
            department.textContent = "Department : " + student.department;

            const cgpa = document.createElement("p");
            cgpa.textContent = "CGPA : " + student.cgpa;

            // Add elements to webpage
            profile.appendChild(heading);
            profile.appendChild(name);
            profile.appendChild(roll);
            profile.appendChild(department);
            profile.appendChild(cgpa);
        });
    </script>

</body>
</html>
```

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Object Instantiation	Define new Student()	Object is created with correct properties	**Pass**
TC-02	Event Trigger	Click Show Profile button	Event listener executes	**Pass**
TC-03	Dynamic Rendering	Verify DOM insertion	Profile data is displayed correctly	**Pass**

9. OUTPUT



10. RESULT

Thus, the Object-Oriented Student Profile Viewer was successfully implemented, demonstrating JavaScript class encapsulation, object instantiation, and dynamic DOM rendering.